
EEEM071 Advanced Topics in Computer Vision and Deep Learning

Lab Week 8

Model Architecture

Dr. Xiatian Zhu
xiatian.zhu@surrey.ac.uk

1 Introduction

Today we learn what is feature extraction, and how to perform feature extraction with various famous architectures such as ResNet and ViT. We will also learn simple MLP and common layers.

Today we learn how to use the `torchvision` package to help us do *data pre-processing and augmentation*. We will also learn how to modify the code section of the data loading in the coursework and the Fashion-MNIST for data augmentation. After modifying it, try run the experiments and observe the performance changes. Note that it might cause slow convergence due to slowing the overfitting process, but most of the time the final performance can be higher (if it is lower, try tuning the learning rate, if not, the augmentation might be too strong/too weak).

2 Feature extraction

Feature extraction is the process of selecting and transforming the most relevant and informative features (or characteristics) from raw data, in order to make it more suitable for further analysis or machine learning tasks. The goal is to reduce the dimensionality of the data while preserving as much useful information as possible.

In image processing, features such as edges, textures, and color histograms can be extracted from an image to represent its content. In deep learning, feature extraction is often done using convolutional neural networks (CNNs), which are designed to automatically learn features from raw input data. The first layers of a CNN are typically responsible for low-level feature extraction, such as edges and textures, while deeper layers learn increasingly abstract and complex features that capture higher-level representations of the input. See Figure 1.

3 Convolutional Neural Network and Vision Transformer

As you have learned from the class, a Convolutional Neural Network (CNN) is a type of deep neural network that is widely used in image recognition and computer vision tasks. CNNs are designed to automatically learn hierarchical representations of visual features from raw input images, without the need for manual feature engineering (meaning a human design feature extraction based on algorithms only).

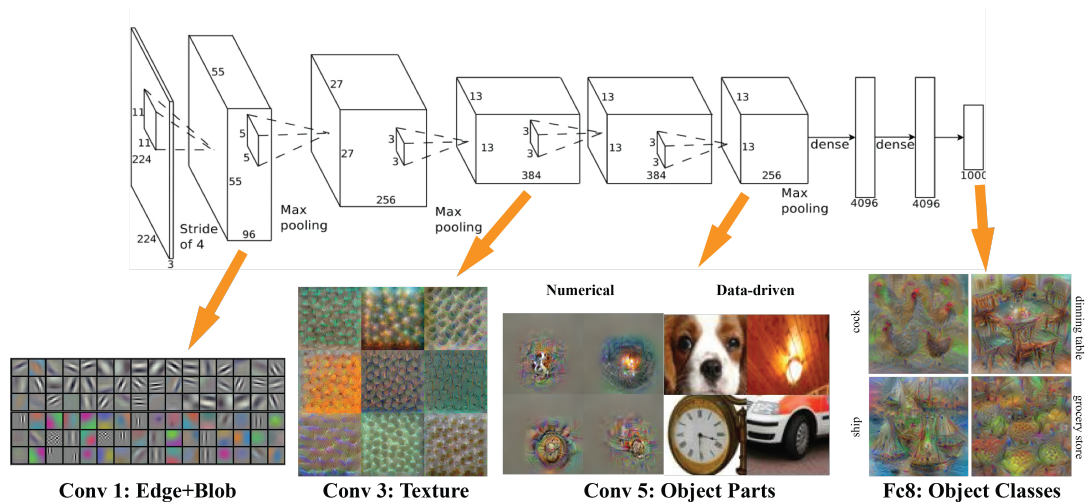


Figure 1: How each layer of a trained AlexNet processes an image. You will see the shallow layer usually detect simple features like edges and corners while the deeper one usually represents complex features like shapes, objects, and scenes. Although this is the case for AlexNet, you can also understand that more complex architecture like ResNet50 might have a similar observation.

The key idea behind CNNs is to apply convolutional filters to the input image, which can detect local patterns or features such as edges, corners, and textures. These filters are learned automatically during the training process, and can be combined in multiple layers to capture more complex and abstract features in the input image.

On the other hand, the Vision Transformer (ViT) is a recent deep learning architecture for image recognition that uses a transformer-based approach, originally developed for natural language processing tasks.

The key idea behind ViT is to break down the input image into small patches, and then treat each patch as a sequence of tokens, similar to how words are represented in natural language processing tasks. These patches are then processed by a series of transformer blocks, which can capture complex relationships and dependencies between the different patches.

Compared to CNNs, ViT has several potential advantages:

1. **More flexible:** Since ViT is based on the transformer architecture, it is more flexible and can potentially handle a wider range of input sizes and resolutions (*i.e.*, it processes a sequence of patches no matter how long the sequence is), without the need for specialized architectures or data augmentation techniques.
2. **More interpretable:** Since the attention mechanism in transformers allows for explicit modeling of relationships between different patches, ViT may be more interpretable and transparent compared to CNNs, which rely on learned convolutional filters.
3. **Potentially higher accuracy:** Some recent studies have shown that ViT can achieve state-of-the-art performance on several benchmark datasets for image recognition, suggesting that it may be a promising alternative to CNNs.

However, ViT also has some potential disadvantages compared to CNNs:

1. Higher computational cost: The transformer-based approach used in ViT requires more computations and memory compared to traditional CNNs, which can make it more challenging to train and deploy on resource-constrained devices.
2. Limited data efficiency: Since ViT is based on a large number of parameters, it may require more labeled data to achieve good performance, compared to smaller and more efficient CNN models.

Overall, ViT represents an exciting new direction in deep learning for image recognition, and has shown promising results on several benchmark datasets. However, it is still a relatively new approach and more research is needed to fully understand its strengths and limitations, as well as how it compares to traditional CNNs on a wider range of tasks and datasets.

4 Image Classification with Pre-Trained Models

In this [link](#), there is a list of available pre-trained models, meaning somebody already trained these models with image classification task so what we can do is to take these trained models as initialization of our task to speed up the training and improve the performance. In Figure 2, we perform simple image classification with a pre-trained ResNet50. In Figure 3, we can simply change the model to ViT-B-16. Please check out the link above to see which model you want to use, there is no rule to choose, you can choose the latest one like [convnext_base](#) but it might be slower, you can choose a moderate one like [mobilenet_v3_small](#) but it can run faster.

You are encouraged to modify the model architecture in the coursework and see whether your modifications can improve performance. You can mention your modifications and observations in your final report. Please refer to the Sec. 8.1 of CourseWork document for more requirements. Note that performance is not all you need. More importantly, exploring as many things as possible, analyzing them, giving insights and thoughts, and finally putting these in the final report in a proper and clear way.

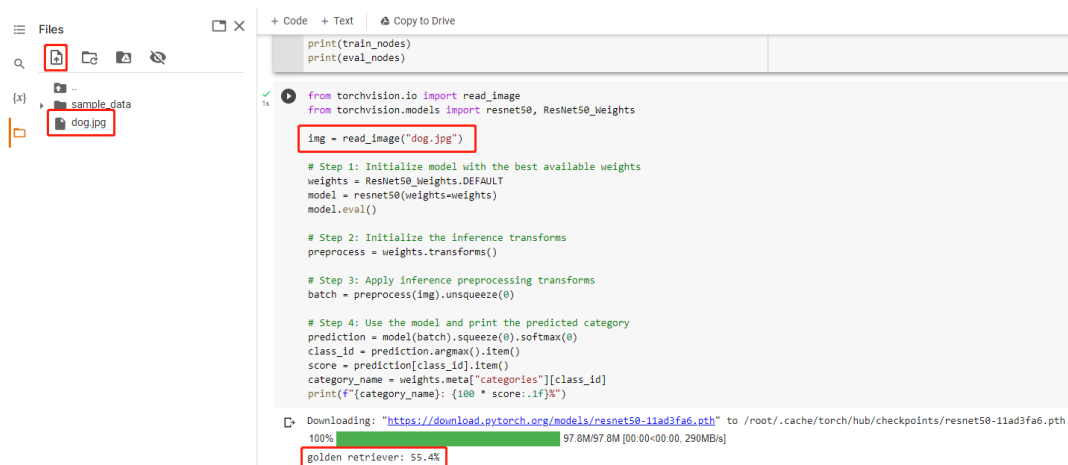


Figure 2: Image Classification with ResNet50. To upload an image, at the side bar, click the “folder” icon, then click the upload button, select your own image and rename the image file name for convenience. In the code, modify `read_image("dog.jpg")` to your own file name. The classification result is shown with probability.

```

print(train_nodes)
print(eval_nodes)

from torchvision.io import read_image
# from torchvision.models import resnet50, ResNet50_Weights
from torchvision.models import vit_b_16, ViT_B_16_Weights

img = read_image("dog.jpg")

# Step 1: Initialize model with the best available weights
weights = ResNet50_Weights.DEFAULT
# model = resnet50(weights=weights)
weights = ViT_B_16_Weights.DEFAULT
model = vit_b_16(weights=weights)
model.eval()

# Step 2: Initialize the inference transforms
preprocess = weights.transforms()

# Step 3: Apply inference preprocessing transforms
batch = preprocess(img).unsqueeze(0)

# Step 4: Use the model and print the predicted category
prediction = model(batch).squeeze(0).softmax(0)
class_id = prediction.argmax().item()
score = prediction[class_id].item()
category_name = weights.meta["categories"][class_id]
print(f"{category_name}: {100 * score:.1f}%")

Downloading: "https://download.pytorch.org/models/vit_b_16-c867db91.pth" to /root/.cache/torch/hub/checkpoints/vit_b_16-c867db91.pth
100% 330M/330M [00:01<00:00, 183MB/s]
golden retriever: 78.8%

```

Figure 3: Image Classification with ViT-B-16. Note the difference to change a model architecture.

```

210     x = self.conv1(x)
211     x = self.bn1(x)
212     x = self.relu(x)
213     x = self.maxpool(x)
214     x = self.layer1(x)
215     x = self.layer2(x)
216     x = self.layer3(x)
217     x = self.layer4(x)
218     return x
219
220 def forward(self, x):
221     f = self.featuremaps(x)
222     v = self.global_avgpool(f)
223     v = v.view(v.size(0), -1)
224
225     if self.fc is not None:
226         v = self.fc(v)
227
228     if not self.training:
229         return v
230
231     y = self.classifier(v)
232
233     if self.loss == {"xent"}:
234         return y
235     elif self.loss == {"xent", "htri"}:
236         return y, v
237     else:
238         raise KeyError(f"Unsupported loss: {self.loss}")
239

```

Figure 4: ResNet forward function.

5 Extra: How to define your custom model for coursework

In the coursework, by default we use pre-trained ResNet50 as our feature extraction network. However, it is different than image classification where we need to modify the default forward function to extract what we want shown in Figure 4. You will see that in training mode, two outputs (*i.e.*, outputs, features) are returned but in testing mode, only one output (*i.e.*, features) is returned. Therefore we need to modify the forward function of the model we used if we want to use other model architectures.

In Figure 7, this is an example of using ViT-B-16 as feature extraction network. We need to first look at the source code of ViT-B-16 at this [link](#). A classifier/head usually takes as input a feature representation

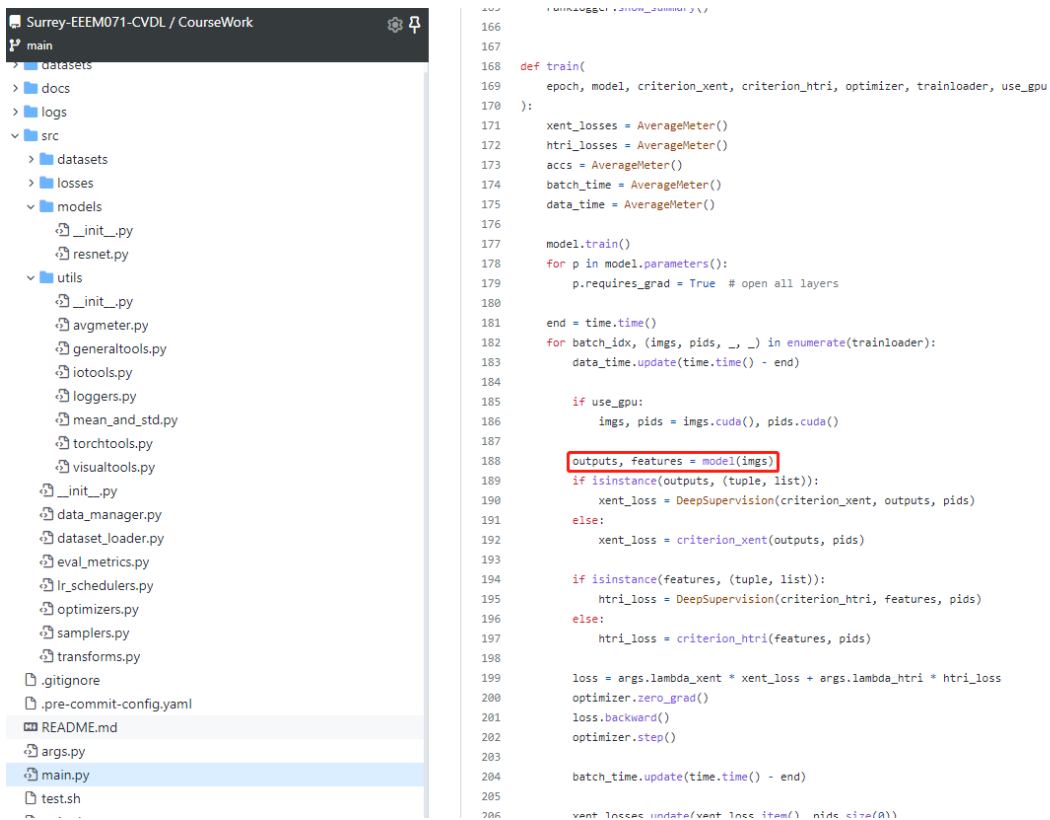
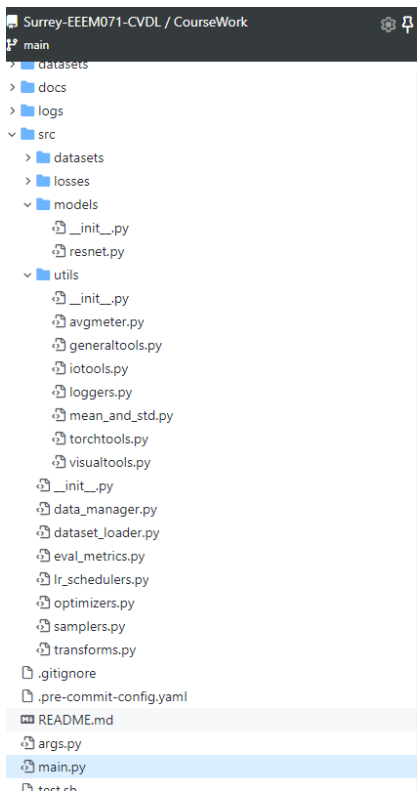


Figure 5: Output when images are forwarded during training.

for classification, so we can easily spot the features of the images, $x = x[:, 0]$. Note that different architectures might have different implementations, so you might need to find out yourself by checking out the source codes.

Finally, in Figure 8, we can replace `models.init_model` with our own custom model as `model = MyCustomModel(num_classes=dm.num_train_pids, loss="xent", "htri")`.



```

230
231
232 def test(
233     model,
234     queryloader,
235     galleryloader,
236     use_gpu,
237     ranks=[1, 5, 10, 20],
238     return_distmat=False,
239 ):
240     batch_time = AverageMeter()
241
242     model.eval()
243
244     with torch.no_grad():
245         qf, q_pids, q_camids = [], [], []
246         for batch_idx, (imgs, pids, camids, _) in enumerate(queryloader):
247             if use_gpu:
248                 imgs = imgs.cuda()
249
250             end = time.time()
251             features = model(imgs)
252             batch_time.update(time.time() - end)
253
254             features = features.data.cpu()
255             qf.append(features)
256             q_pids.extend(pids)
257             q_camids.extend(camids)
258         qf = torch.cat(qf, 0)
259         q_pids = np.asarray(q_pids)
260         q_camids = np.asarray(q_camids)
261
262         print(
263             "Extracted features for query set, obtained {}-by-{} matrix".format(
264                 qf.size(0), qf.size(1)
265             )
266         )
267
268         gf, g_pids, g_camids = [], [], []
269         for batch_idx, (imgs, pids, camids, _) in enumerate(galleryloader):
270             if use_gpu:

```

Figure 6: Output when images are forwarded during testing.

```

import torch
import torch.nn as nn
from torchvision.models import vit_b_16, ViT_B_16_Weights  # you can import any model you want as long as it is listed in the link

class MyCustomModel(nn.Module):
    def __init__(self,
                  num_classes,
                  loss={"xent", "htri"},
                  pretrained=True,
                  **kwargs):  # setup your own arguments or keyword arguments if needed
        super().__init__()  # remember always write this line --- this is for pytorch magic

        # note, this is new implementation of how to load pre-trained weights
        weights = ViT_B_16_Weights.DEFAULT
        model = vit_b_16(weights=weights)

        # assign this loaded model to self.model
        self.model = model

        # print(model) # you can always print the model, check out the classifier/head layer of the model

        feature_dim = 768  # from the print result i find that the feature dim of ViT Base is 768

        self.loss = loss
        self.num_classes = num_classes
        self.classifier = nn.Linear(feature_dim, num_classes)  #

        # model = vit_b_16(pretrained=True)  # this is how to load back then

    def forward(self, x):  # always setup the forward function, this function literally means what it should do during a forward pass
        # check out this link to understand the forward function of a ViT
        # https://github.com/pytorch/vision/blob/b403bfc771e0caf31efd06d43860b09004f4ac61/torchvision/models/vision\_transformer.py#L289

        #####
        # Reshape and permute the input tensor
        x = self.model._process_input(x)  # note that it is self.model instead of self, because we are accessing "self.model"
        n = x.shape[0]

def forward(self, x):  # always setup the forward function, this function literally means what it should do during a forward pass
    # check out this link to understand the forward function of a ViT
    # https://github.com/pytorch/vision/blob/b403bfc771e0caf31efd06d43860b09004f4ac61/torchvision/models/vision\_transformer.py#L289

    #####
    # Reshape and permute the input tensor
    x = self.model._process_input(x)  # note that it is self.model instead of self, because we are accessing "self.model"
    n = x.shape[0]

    # Expand the class token to the full batch
    batch_class_token = self.model.class_token.expand(n, -1, -1)
    x = torch.cat([batch_class_token, x], dim=1)

    x = self.model.encoder(x)

    # Classifier "token" as used by standard language architectures
    v = x[:, 0]
    #####

    if not self.training:
        return v

    y = self.classifier(v)

    if self.loss == {"xent"}:
        return y
    elif self.loss == {"xent", "htri"}:
        return y, v
    else:
        raise KeyError(f"Unsupported loss: {self.loss}")

```

Figure 7: Defining custom model with ViT-B-16. Note the similarity and differences with the source code in coursework.

```

58     print("Initializing image data manager")
59     dm = ImageDataManager(use_gpu, **dataset_kwargs(args))
60     trainloader, testloader_dict = dm.return_data loaders()
61
62     print(f"Initializing model: {args.arch}")
63     model = models.init_model(
64         name=args.arch,
65         num_classes=dm.num_train_pids,
66         loss={"xent", "htri"},
67         pretrained=not args.no_pretrained,
68         use_gpu=use_gpu,
69     )
70     print("Model size: {:.3f} M".format(count_num_param(model)))
71
72     if args.load_weights and check_isfile(args.load_weights):
73         load_pretrained_weights(model, args.load_weights)

```

Figure 8: Code where model is declared.